

CO4: MACHINE LEARNING

SCENARIO-BASED ASSESSMENT

Question 6

Loan Approval Prediction Using Logistic Regression

Submission Detail	Enter Your Details
Name	_____
Registration Number	_____
Course Code	_____
Course Outcome	CO4
Question Number	6
Algorithm	Logistic Regression
Assessment Type	Individual Assessment

Code of Conduct Declaration

I declare that this assessment is my individual work and that I will comply with the academic integrity and Code of Conduct requirements of the institution.

Signature: _____ Date: _____

1. Scenario Understanding and Objective

A bank wants to predict whether a loan application should be approved or rejected using applicant financial and employment information. The target is binary: Approved = 1 (approved) and Approved = 0 (rejected).

Objective: Build a Logistic Regression classification model, predict the test samples and a new applicant, evaluate the model using the required classification metrics, and interpret the result for the banking scenario.

Assessment Rubric Coverage

Criterion	Weight	How this report addresses it
Understanding of Scenario	20%	Explains the banking problem, target classes and objective.
Identification of Key Issues	20%	Identifies input features, target variable, train/test split and data-size limitation.
Application of Concepts	25%	Implements dataset creation, split, Logistic Regression, prediction and all required metrics.
Decision Making	20%	Explains the new-applicant decision and discusses limitations.
Clarity of Response	15%	Uses structured sections, tables, code, results and conclusion.

2. Task (a): Dataset Creation

The dataset contains eight loan applications and four independent variables.

Applicant	Income	Credit Score	Existing Loan	Employment Years	Approved
A1	25,000	580	200,000	1	0
A2	30,000	600	180,000	2	0
A3	45,000	680	120,000	3	1
A4	60,000	750	80,000	5	1
A5	70,000	800	50,000	6	1
A6	28,000	590	220,000	1	0
A7	55,000	720	100,000	4	1
A8	35,000	620	160,000	2	0

3. Task (b): Independent and Dependent Variables

Type	Variable	Description
Independent (X)	Income	Applicant income
Independent (X)	CreditScore	Applicant credit score
Independent (X)	ExistingLoan	Existing loan amount
Independent (X)	EmploymentYears	Years of employment
Dependent (y)	Approved	1 = approved, 0 = rejected

4. Task (c): Train-Test Split

The dataset is split into 75% training data and 25% testing data. `random_state=42` makes the split reproducible and `stratify=y` preserves class proportions as far as possible.

Set	Applicants	Records
Training	A3, A2, A4, A7, A1, A8	6
Testing	A6, A5	2

5. Task (d): Logistic Regression

The model is trained using `LogisticRegression(max_iter=1000)`. Logistic Regression estimates the probability of the binary target and produces class predictions.

6. Task (e): Test Data Prediction

Applicant	Actual	Predicted	Result
A6	0 (Rejected)	0 (Rejected)	Correct
A5	1 (Approved)	1 (Approved)	Correct

Both test observations are correctly classified.

7. Task (f): New Applicant Prediction

Feature	New Applicant
Income	50,000
Credit Score	700
Existing Loan	100,000
Employment Years	4

Predicted class: 1 — LOAN APPROVED. The fitted model gives an approval probability extremely close to 1.0000 for this small dataset. This probability should not be treated as a reliable real-world risk probability.

8. Task (g): Model Evaluation

Metric	Value	Interpretation
Accuracy	1.00 (100%)	2 of 2 test samples correctly classified.
Confusion Matrix	[[1, 0], [0, 1]]	TN=1, FP=0, FN=0, TP=1.
Precision	1.00 (100%)	All predicted approvals were actually approved.
Recall	1.00 (100%)	All actual approvals in the test set were identified.
F1-Score	1.00 (100%)	Precision and recall are both 1.00.

9. Task (h): Why Logistic Regression is Suitable

- The target has two classes, approved (1) and rejected (0), so this is a binary classification problem.
- It estimates the probability of class membership, which is useful for decision-support.
- It is computationally efficient and relatively easy to interpret.
- It provides a strong baseline for banking classification before using more complex algorithms.

10. Complete Python Program

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, confusion_matrix,
    precision_score, recall_score, f1_score
)

# 1. Create the dataset
data = {
    "Applicant": ["A1", "A2", "A3", "A4", "A5", "A6", "A7", "A8"],
    "Income": [25000, 30000, 45000, 60000, 70000, 28000, 55000, 35000],
    "CreditScore": [580, 600, 680, 750, 800, 590, 720, 620],
    "ExistingLoan": [200000, 180000, 120000, 80000, 50000, 220000, 100000, 160000],
    "EmploymentYears": [1, 2, 3, 5, 6, 1, 4, 2],
    "Approved": [0, 0, 1, 1, 1, 0, 1, 0]
}
df = pd.DataFrame(data)

print("DATASET")
print(df)

# 2. Identify independent and dependent variables
x = df[["Income", "CreditScore", "ExistingLoan", "EmploymentYears"]]
y = df["Approved"]

# 3. Split the dataset: 75% training, 25% testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

print("\nTraining rows:", X_train.index.tolist())
print("Testing rows:", X_test.index.tolist())

# 4. Implement Logistic Regression
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# 5. Predict approval status for test data
y_pred = model.predict(X_test)
print("\nActual test values:", y_test.values)
print("Predicted test values:", y_pred)

# 6. Predict the new applicant
new_applicant = pd.DataFrame(
    [[50000, 700, 100000, 4]],
    columns=["Income", "CreditScore", "ExistingLoan", "EmploymentYears"]
)

new_prediction = model.predict(new_applicant)
new_probability = model.predict_proba(new_applicant)

print("\nNew applicant prediction:")
print("Status:", "Approved" if new_prediction[0] == 1 else "Rejected")
print("Probability of rejection:", new_probability[0][0])
print("Probability of approval:", new_probability[0][1])

# 7. Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("\nMODEL EVALUATION")
print("Accuracy:", accuracy)
print("Confusion Matrix:\n", cm)
print("Precision:", precision)
print("Recall:", recall)
```

```
print("F1-Score:", f1)
```

11. Interpretation and Decision Making

The model correctly predicts both test observations. It also predicts that the new applicant (income 50,000, credit score 700, existing loan 100,000 and four employment years) should be approved.

The new applicant has a comparatively stronger profile than the rejected examples in the given dataset: higher income than several rejected applicants, a higher credit score, and a lower existing-loan amount than several rejected applicants.

Important limitation: The dataset has only eight observations and the test set has only two observations. Therefore, the 100% scores should not be considered proof of real-world model performance. A larger representative dataset and additional validation are required before deployment.

12. Key Issues and Limitations

- Very small dataset: only 8 observations.
- Very small test set: only 2 observations.
- Only four applicant factors are represented.
- Production use would require broader validation, fairness checks and domain review.
- The reported metrics are specific to the selected train-test split.

13. Conclusion

Logistic Regression was successfully implemented for binary loan approval classification. The model achieved 100% accuracy, precision, recall and F1-score on the selected test split, with confusion matrix $\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$. The new applicant was predicted as approved. Logistic Regression is appropriate because the outcome is binary and the method provides an interpretable probability-based classification approach. However, the extremely small dataset means the performance is illustrative rather than evidence of production-level accuracy.

14. Submission Checklist

- Enter name, registration number, course code and question number on the first page.
- Attach the Python source file (.py or .ipynb).
- Include dataset creation and variable identification.
- Include train-test split and Logistic Regression implementation.
- Include test predictions and new-applicant prediction.
- Include accuracy, confusion matrix, precision, recall and F1-score.
- Include interpretation, decision and conclusion.
- Complete and sign the Code of Conduct declaration.
- If faculty requires screenshots, execute the code in Jupyter/Colab and insert screenshots of the outputs before final submission.